Chapter 16. Processing JSON Data

16.0 Introduction

JSON, or JavaScript Object Notation, is all of the following:

- A simple, lightweight data interchange format
- A simpler, lighter alternative to XML
- Easy to generate with `println()` or with one of several APIs
- Recognized directly by the JavaScript parser in all browsers, server-side node implementations, etc.
- Supported with add-on frameworks for all common languages (Java, C/C++, Perl, Ruby, Python, Lua, Erlang, Haskell, to name a few); a ridiculously long list of supported languages (including two dozen parsers for Java alone) is on the JSON home page

A basic JSON message might look like Example 16-1.

Example 16-1. ***json/src/main/resources/json/softwareinfo.json***

```
{
  "name": "robinparse",
  "version": "1.2.3",
  "description": "Another Parser for JSON",
  "className": "RobinParse",
  "contributors": [
      "Robin Smythe",
      "Jon Jenz",
      "Jan Ardann"
   ]
}
```

As you can see, the syntax is simple, nestable, and amenable to human inspection.

The JSON home page provides a concise summary of JSON syntax. There are two kinds of structure: JSON objects (maps) and JSON arrays (lists). JSON *objects* are sets of name and value pairs, which can be represented in Java either as a `java.util.Map` *or* as the properties of a Java object. For example, the fields of a `LocalDate` (see Recipe 6.1) object for April 1, 2025, might be represented like this:

```
{
    "year": 2025,
    "month": 4,
    "day" : 1
}
```

JSON *arrays* are ordered lists, represented in Java either as arrays or as `java.util.List`s. A list of two dates might look like this:

```
{
    [{
        "year": 2025,
        "month": 4,
        "day" : 1
    },{
        "year": 2025,
        "month": 5,
        "day" : 15
    }]
}
```

JSON is not whitespace-sensitive, so the preceding could also be written, with some loss of human readability but no loss of information or functionality, as this:

```
{[{"year":2025,"month":4,"day":1},{"year":2025,"month":5,"day":15}]}
```

Hundreds of parsers have, I'm sure, been written for JSON; a long list is at the middle of the JSON.org website. A few that come to mind for Java include the following:

stringtree.org

Very small and lightweight

org.json parser

Widely used because it's free and has a good domain name

jackson.org parser

Widely used because it's very powerful and used with Spring Framework and with JBoss RESTEasy and WildFly

jakarta.json a.k.a jsonp

The "official" Java standard

This chapter shows several ways of processing JSON data using some of the various APIs just listed. The official `jakarta.json` API is part of the Jakarta Enterprise API, so it is primarily used on the server side. This API uses some names that are also used by the `org.json` API, but not enough to be considered compatible; you can't easily use both in the same application.

16.1 Generating JSON Directly

Problem

You want to generate JSON without bothering to use an API.

Solution

Get the data you want, and use `println()` or `String.format()` as appropriate.

Discussion

If you are careful, you can generate JSON data yourself. For the utterly trivial cases, you can just use `PrintWriter.println()` or `String.format()`. For significant volumes, however, it's usually better to use one of the APIs.

This code prints the year, month, and date from a `LocalTime` object (see Recipe 6.1). Some of the JSON formatting is delegated to the `toJson()` method:

```
/**
 * Convert an object to JSON, not using any JSON API.
 * BAD IDEA - should use an API!
 */
public class LocalDateToJsonManually {
```

```
  private static final String OPEN = "{";
  private static final String CLOSE = "}";

  public static void main(String[] args) {
    LocalDate dNow = LocalDate.now();
    System.out.println(toJson(dNow));
  }

  public static String toJson(LocalDate dNow) {
    StringBuilder sb = new StringBuilder();
    sb.append(OPEN).append("\n");
    sb.append(jsonize("year", dNow.getYear()));
    sb.append(jsonize("month", dNow.getMonth()));
    sb.append(jsonize("day", dNow.getDayOfMonth()));
    sb.append(CLOSE).append("\n");
    return sb.toString();
  }

  public static String jsonize(String key, Object value) {
    return String.format("\"%s\": \"%s\",\n", key, value);
  }
}
```

Of course, this is an extremely basic example. For anything more involved, or for the common scenario of having to parse JSON objects, using one of the frameworks will be easier on your nerves. That said, for some applications, the DIY approach may be faster. Time it and see!

16.2 Parsing and Writing JSON with Jackson

Problem

You want to read and/or write JSON using a full-function JSON API.

Solution

Use Jackson, the full-blown JSON API.

Discussion

Jackson provides many ways to work with JSON. For simple cases, you can have POJO (Plain Old Java Objects) converted to/from JSON more or less automatically, as is illustrated in Example 16-2.

You need the following three dependencies for full use of Jackson; later versions may be available when you read this:

```
com.fasterxml.jackson.core:jackson-core:2.13.0
com.fasterxml.jackson.core:jackson-annotations:2.13.0
com.fasterxml.jackson.core:jackson-databind:2.13.4.2
```

Example 16-2. json/src/main/java/json/ReadWriteJackson.java (reading and writing POJOs with Jackson)

```
import java.io.IOException;
import com.fasterxml.jackson.databind.ObjectMapper;

public class ReadWriteJackson {


   public static void main(String[] args) throws IOException {
      ObjectMapper mapper = new ObjectMapper();            ❶

      String jsonInput =                                   ❷
            "{\"id\":0,\"firstName\":\"Robin\",\"lastName\":\"Wilson\"}";
      Person q = mapper.readValue(jsonInput, Person.class);
      System.out.println("Read and parsed Person from JSON: " + q);

      Person p = new Person(0, "Roger", "Rabbit");         ❸
      System.out.print("Person object " + p +" as JSON = ");
      mapper.writeValue(System.out, p);
   }

   record Person(int id, String firstName, String lastName){   ❹
   @Override
   public String toString() {
    StringBuilder sb = new StringBuilder();
    if (firstName != null)
      sb.append(firstName).append(' ');
    if (lastName != null)
      sb.append(lastName);
```

```
    if (sb.length() == 0)
      sb.append("NO NAME");
    return sb.toString();
  }
 }
}
```

❶ Create a Jackson `ObjectMapper` that can map POJOs to/from JSON.

❷ Map the string `jsonInput` into a `Person` object with one call to `readValue()`.

❸ Convert the `Person` object `p` into JSON with one call to `writeValue()`.

❹ This class has its own `Person` type definition, a `record` with only three fields and a useful `toString()` method.

Running this example produces the following output:

```
Read and parsed Person from JSON: Robin Wilson
Person object Roger Rabbit as JSON = {"id":0,"firstName":"Roger",
      "lastName":"Rabbit","name":"Roger Rabbit"}
```

As another example, this code reads the example file that opened this chapter (which happens to have been a description of a JSON parser):

```
public class SoftwareParseJackson {
 final static String FILE_NAME = "/json/softwareinfo.json";

 public static void main(String[] args) throws Exception {
  ObjectMapper mapper = new ObjectMapper();                    ❶

  InputStream jsonInput =
   SoftwareParseJackson.class.getResourceAsStream(FILE_NAME);      ❷
  if (jsonInput == null) {
   throw new NullPointerException("can't find " + FILE_NAME);
  }
  SoftwareInfo sware = mapper.readValue(jsonInput, SoftwareInfo.class); ❸
  System.out.println(sware);
 }

}
```

❶ The `ObjectMapper` does the actual parsing of the JSON input.

❷ Reading the file using `getResourceAsStream()` (see Recipe 10.16).

❸ Converting the JSON string into a `domain.SoftwareInfo` object.

`SoftwareInfo` is a simple `record` type; notice the declaration `List<String>` for the contributors, which Jackson is quite capable of handling:

```
public record SoftwareInfo(String name,
 String version, String description,
 String className, List<String> contributors) {

 @Override
 public String toString() {
  return String.format("Software: %s (%s) by %s", name, version, contributors);
 }
}
```

Running this example produces the following output:

```
Software: robinparse (1.2.3) by [Robin Smythe, Jon Jenz, Jan Ardann]
```

Of course there are situations where the mapping gets more involved; for this purpose, Jackson provides a set of annotations to control the mapping. But the default mapping is pretty good!

There is also a streaming API for Jackson, and even a large array of data format converters for formats other than JSON. Refer to the Jackson website for details.

16.3 Parsing and Writing JSON with org.json

Problem

You want to read/write JSON using a midsized, widely used JSON API.

Solution

Consider using the org.json API , also known as JSON-Java; it's widely used and is also used in Android.

Discussion

The org.json package is not as advanced as Jackson, nor as high level; it makes you think and work in terms of the underlying JSON abstractions instead of at the Java code level. For example, here is the org.json version of reading the software description from the opening of this chapter:

```
public class SoftwareParseOrgJson {
   final static String FILE_NAME = "/json/softwareinfo.json";

   public static void main(String[] args) throws Exception {

      InputStream jsonInput =
         SoftwareParseOrgJson.class.getResourceAsStream(FILE_NAME);
      if (jsonInput == null) {
         throw new NullPointerException("can't find" + FILE_NAME);
      }
      JSONObject obj = new JSONObject(new JSONTokener(jsonInput));     ❶
      System.out.println("Software Name: " + obj.getString("name"));   ❷
      System.out.println("Version: " + obj.getString("version"));
      System.out.println("Description: " + obj.getString("description"));
      System.out.println("Class: " + obj.getString("className"));
      JSONArray contribs = obj.getJSONArray("contributors");          ❸
      for (int i = 0; i < contribs.length(); i++) {                   ❹
         System.out.println("Contributor Name: " + contribs.get(i));
      }
   }

}
```

❶ Create the `JSONObject` from the input.

❷ Retrieve individual `String` fields.

❸ Retrieve the `JSONArray` of contributor names.

❹

`org.json.JSONArray` doesn't implement `Iterable`, so you can't use a `forEach` loop.

Running it produces the expected output:

```
Software Name: robinparse
Version: 1.2.3
Description: Another Parser for JSON
Class: RobinParse
Contributor Name: Robin Smythe
Contributor Name: Jon Jenz
Contributor Name: Jan Ardann
```

`JSONObject` and `JSONArray` use their `toString()` method to produce (correctly formatted) JSON strings, like this:

```
public class WriteOrgJson {
  public static void main(String[] args) {
    JSONObject jsonObject = new JSONObject();
    jsonObject.put("Name", "robinParse").    ❶
      put("Version", "1.2.3").
      put("Class", "RobinParse");
    String printable = jsonObject.toString(); ❷
    System.out.println(printable);
  }
}
```

❶ Nice that it offers a fluent API to allow chaining of method calls.

❷ `toString()` converts to textual JSON representation.

Running this produces the following:

```
{"Name":"robinParse","Class":"RobinParse","Version":"1.2.3"}
```

See Also

The org.json library code, including its Javadoc documentation, is available online at the JSON-java GitHub repository (under the name JSON-Java to differentiate it from the other packages).

16.4 Parsing and Writing JSON with JSON-B

Problem

You want to read/write JSON using a midsized, standards-conforming JSON API.

Solution

Consider using JSON-B, the new Java standard (JSR-367).

Discussion

The JSON-B (JSON Binding) API is designed to make it easy to read/write Java POJOs. This is neatly illustrated by the code in Example 16-3.

Example 16-3. json/src/main/java/json/ReadWriteJsonB.java (reading/writing JSON with JSON-B)

```
public class ReadWriteJsonB {

  public static void main(String[] args) throws IOException {

   Jsonb jsonb = JsonbBuilder.create(); ❶

   // Read
   String jsonInput =                   ❷
    "{\"id\":0,\"firstName\":\"Robin\",\"lastName\":\"Williams\"}";
   Person rw = jsonb.fromJson(jsonInput, Person.class);
   System.out.println(rw);

   String result = jsonb.toJson(rw);    ❸
   System.out.println(result);
  }
}
```

❶ Create a `Jsonb` object, your gateway to JSON-B services.

❷ Obtain a JSON string, and convert it to a Java object using `jsonb.fromJson()`.

❸ Convert a `Person` object back to a JSON string using the inverse `jsonb.toJson()`.

Note that the methods are sensibly named and that no annotations are needed on the Java entity class to make this work. However, there is an API that allows us to customize it. For example, the `fullName` property is really just a convenience for concatenating the first name and last name with a space in between. As such, it's completely redundant and does not need to be transmitted over a JSON network stream. However, running the program produces this output:

```
{"firstName":"Robin","fullName":"Robin Williams","id":0,"lastName":"Williams"}
```

We need only add the `@JsonbTransient` annotation to the `getFullName()` accessor in the `Person` class to eliminate the redundancy; running the program now produces this smaller output:

```
{"firstName":"Robin","id":0,"lastName":"Williams"}
```

See Also

As with most other JSON APIs, there is full support for customization, ranging from the simple annotation shown here up to writing complete custom serializer/deserializer helpers. See the JSON-B spec page and this longer tutorial online.

16.5 Finding JSON Elements with JSON Pointer

Problem

You have a JSON document and want to extract only selected values from it.

Solution

Use javax.json's implementation of JSON Pointer, the standard API for extracting selected elements from JSON.

Discussion

The Internet Standard RFC 6901 spells out in detail the syntax for JSON Pointer, a language-independent syntax for matching elements in JSON documents. Inspired by the XML syntax XPath, JSON Pointer is a bit simpler than XPath because of JSON's inherent

simplicity. Basically, a JSON Pointer is a string that identifies an element (either simple or array) within a JSON document. The javax.json package provides an object model API somewhat similar to the XML DOM API for Java, letting you create immutable objects to represent objects (via `JsonObjectBuilder` and `JsonArrayBuilder`) or to read them from JSON string format via a `Reader` or `InputStream`.

JSON Pointers begin with a `/` (inherited from XPath), followed by the name of the element or subelement we want to look for. Suppose we extend our `Person` example from Example 16-3 to add an array of roles the comedian played, looking like this:

```
{"firstName":"Robin","lastName":"Williams",
      "age": 63,"id":0,
      "roles":["Mork", "Mrs. Doubtfire", "Patch Adams"]}
```

Then the following JSON Pointers should generate the given matches:

```
/firstName => Robin
/age => 63
/roles => ["Mork","Mrs. Doubtfire","Patch Adams"]
/roles/1 => "Mrs. Doubtfire"
```

The program in Example 16-4 demonstrates this. And its actual output is similar to what's expected:

```
/firstName => Robin
/age => 63; a org.eclipse.parsson.JsonNumberImpl$JsonIntNumber
/roles => ["Mork","Mrs. Doubtfire","Patch Adams"]
JsonArray roles.get(1) => "Mrs. Doubtfire"
/roles/1 => "Mrs. Doubtfire"
```

Example 16-4. json/src/main/java/json/JsonPointerDemo.java

```
public class JsonPointerDemo {

  public static void main(String[] args) {
    String jsonPerson = """
     {"firstName":"Robin","lastName":"Williams",
      "age": 63,
      "id":0,
      "roles":["Mork", "Mrs. Doubtfire", "Patch Adams"]
```

```
    }
    """;

    System.out.println("Input: " + jsonPerson);

    JsonReader rdr =
        Json.createReader(new StringReader(jsonPerson)); ❶
    JsonStructure jsonStr = rdr.read();
    rdr.close();

    JsonPointer jsonPointer = Json.createPointer("/firstName"); ❷
    JsonString jsonString = (JsonString)jsonPointer.getValue(jsonStr);
    String firstName = jsonString.getString();
    System.out.println("/firstName => " + firstName);

    JsonNumber num =                                    ❸
        (JsonNumber) Json.createPointer("/age").getValue(jsonStr);
    System.out.println("/age => " + num + "; a " + num.getClass().getName());

    jsonPointer = Json.createPointer("/roles");         ❹
    JsonArray roles = (JsonArray) jsonPointer.getValue(jsonStr);
    System.out.println("/roles => " + roles);
    System.out.println("JsonArray roles.get(1) => " + roles.get(1));

    jsonPointer = Json.createPointer("/roles/1");       ❺
    jsonString = (JsonString)jsonPointer.getValue(jsonStr);
    System.out.println("/roles/1 => " + jsonString);
  }
}
```

❶ Create the `JsonStructure`, the gateway into this API, from a `JsonReader`, using a `StringReader`.

❷ Create a JSON Pointer for the `firstName` element, and get the `JsonString` from the element's value. Since `getValue()` will throw an exception if the element is not found, use `jsonPointer.containsValue(jsonStr)` to check first, if you're not sure if the element will be found.

❸ Same for `age`, but using more fluent syntax. If you print the class name for the match in `/age`, it will report an implementation-specific implementation class, such as `org.glassfish.json.JsonNumberImpl$JsonIntNumber`. Change the age in the XML from 63 to 63.5 and it will print a class with `BigDecimal` in its name. Either way, calling `toString()` on this object will return just the numeric value.

❹ In the JSON file, `roles` is an array. Thus, getting it using a JSON Pointer should return a `JsonArray` object, so we cast it to a reference of that type. This behaves somewhat like an immutable `List` implementation, so we use the `get()` method. JSON array indices start at zero, as in Java.

❺ Retrieve the same array element directly, using a pattern with `/1` to mean the numbered element in the array.

It is possible (but fortunately not common) for a JSON element name to contain special characters such as a slash. Most characters are not special to JSON Pointer, but to match a name containing a slash (/), the slash must be entered as `~1`, and since that makes the tilde (~) special, tilde characters must be entered as `~0`. Thus if the Person JSON file had an element like `"ft/pt/~"`, you would look for it with `Json.createPointer("/ft~1pt~1~0");`.

See Also

The JSON Pointer API has additional methods that let you modify values and add/remove elements. The offical home page for javax.json, which includes JSON Pointer, is on the Jakarta EE website. That page also links to the Javadoc for javax.json.

Many APIs exist for Java. Jackson is the biggest and most powerful; org.json, `javax.json`, and JSON-B are in the middle; and StringTree (which I didn't give an example of because it doesn't have a Maven Artifact available) is the smallest. For a list of these and other JSON APIs, not just for Java, consult the JSON home page and scroll past the syntax summary.